

SAE 2.01

Documentation Technique

Application Flask — Données de Santé Ameli

BUT Informatique — Groupe B

LIN,JAMAIN et MONAR

Racine du projet

-config.py :

Ce fichier charge les variables d'environnement depuis un fichier .env et centralise toute la configuration de l'application. Les paramètres sensibles (identifiants BDD, clé secrète) ne sont jamais codés en dur.

- **DB_USER, DB_PASSWORD, DB_HOST, DB_NAME** : attributs de classe lisant les variables d'environnement correspondantes via `os.getenv()`.
- **SECRET_KEY** : clé secrète Flask utilisée pour la gestion des sessions. Valeur par défaut : 'dev-secret-changez-moi'.
- **db_url()** : méthode de classe qui construit et retourne l'URL de connexion MySQL au format `mysql+pymysql://user:password@host/dbname`. Elle ne reçoit aucun paramètre et utilise les attributs de la classe.

-db.py :

Ce fichier initialise le moteur SQLAlchemy unique de l'application et fournit une fabrique de sessions. L'option `pool_recycle=3600` permet d'éviter les coupures de connexion MySQL après une longue période d'inactivité.

- **engine** : moteur SQLAlchemy unique créé depuis l'URL fournie par `Config.db_url()`. Partagé par toute l'application.
- **Session** : fabrique de sessions liée au moteur. Chaque requête HTTP doit instancier sa propre session via `Session()` et la fermer dans un bloc `finally` pour libérer la connexion.

-app.py :

Point d'entrée principal de l'application. Ce fichier instancie Flask, charge la configuration, enregistre tous les Blueprints, crée les tables manquantes et définit les gestionnaires d'erreurs globaux.

- **inject_base_url()** : context processor Flask décoré avec `@app.context_processor`. Il injecte la variable `BASE_URL` dans tous les templates Jinja2 pour gérer le préfixe d'URL lors d'un déploiement en sous-répertoire. Ne reçoit aucun paramètre et retourne un dictionnaire `{ 'BASE_URL': str }`.
- **page_non_trouvee(e)** : gestionnaire d'erreur HTTP 404. Reçoit l'exception Flask en paramètre et retourne le template `erreur.html` avec le code 404 et un message explicatif. Statut HTTP de la réponse : 404.
- **erreur_serveur(e)** : gestionnaire d'erreur HTTP 500. Reçoit l'exception Flask en paramètre et retourne le template `erreur.html` avec le code 500 et un message d'erreur générique. Statut HTTP de la réponse : 500.

-wsgi.py :

Point d'entrée pour le serveur WSGI de production (Alwaysdata). Ce fichier encapsule l'application Flask dans un middleware de préfixe d'URL et expose la variable application attendue par le serveur.

- **class PrefixMiddleware** : middleware WSGI qui retire un préfixe d'URL des chemins entrants. Nécessaire lorsque l'application est déployée dans un sous-dossier (exemple : `/monapp/`).
 - **__init__(self, app, prefix="")** : reçoit l'application Flask à encapsuler et le préfixe d'URL à retirer (vide par défaut). Stocke les deux en attributs d'instance.
 - **__call__(self, environ, start_response)** : intercept chaque requête WSGI. Si un préfixe est configuré et que `PATH_INFO` commence par ce préfixe, le retire de

PATH_INFO et le place dans SCRIPT_NAME avant de transmettre la requête à Flask.
Reçoit le dictionnaire d'environnement WSGI et le callback start_response.

- **application** : variable exposant l'objet Flask après application du middleware. C'est cette variable que le serveur WSGI Alwaysdata utilise pour démarrer l'application.

models/

-dimensions.py :

Ce fichier définit les modèles SQLAlchemy (ORM) mappés sur les tables de référence de la base de données. Ces nomenclatures sont utilisées en lecture seule pour enrichir les filtres et les résultats affichés.

- **class Region** : modèle ORM représentant une région administrative française (table : region). Possède les colonnes id, code et libelle, ainsi qu'une relation vers ses départements via departements.
- **class Departement** : modèle ORM représentant un département (table : departement). Possède les colonnes id, code, libelle et region_id (clé étrangère vers Region). Possède également une relation vers son objet Region parent.
- **class ProfessionSante** : modèle ORM listant les professions de santé disponibles dans l'API Ameli (table : profession_sante). Colonnes : id et libelle.
- **class PostePrescription** : modèle ORM représentant un poste de prescription, c'est-à-dire un type de médicament prescrit (table physique : type_prescription). Colonnes : id et libelle.

-utilisateur.py :

Ce fichier définit le modèle ORM utilisateur qui représente la table utilisateur dans la base de données qui contient les identifiants et mot de passe des utilisateurs inscrits sur le site. Il s'appuie sur SQLAlchemy pour faire correspondre une classe python à une table de la base de données.

- **class Utilisateur(Base)** : classe qui représente la table utilisateur dans la base de données.
 - **Id** : clé primaire auto-incrémenté
 - **Identifiant** : chaîne de caractère qui représente l'identifiant d'un utilisateur, champ obligatoire, doublon interdit
 - **mot_de_passe_hache** : chaîne de caractère qui représente le mot de passe haché d'un utilisateur, champ obligatoire

services/

-ameli_api.py :

Ce fichier centralise tous les accès à l'API publique data.ameli.fr. Il gère les requêtes HTTP, le cache mémoire, les retries automatiques et l'agrégation de certaines données.

Variables globales :

- **_cache** : cache mémoire global de type TTLCache avec un TTL de 24 h (maxsize : 512 entrées), suffisant pour toutes les combinaisons profession × territoire × année × dataset. Les données ameli.fr étant publiées annuellement, 24 h est une valeur adaptée.
- **_cache_lock** : verrou threading (Lock) permettant de sécuriser l'accès concurrent au cache entre plusieurs requêtes simultanées.
- **_cache_key(dataset, params)** : fonction qui reçoit le nom du dataset et un dictionnaire de paramètres de requête. Elle génère une clé de cache déterministe via hashlib.sha256 à partir de la sérialisation JSON des paramètres triés. Retourne une chaîne hexadécimale de 64 caractères.

Attributs de classe AmeliAPI :

- **BASE_URL** : URL racine de l'API Explore v2.1 de data.ameli.fr.
- **_CORRESPONDANCES_HONORAIRES** : dictionnaire d'harmonisation de certains libellés de professions, dont les noms diffèrent entre le dataset honoraires et les autres datasets. Utilisé par les méthodes d'honoraires pour corriger le libellé avant d'envoyer la requête.
- **_PAGE_MAX** : taille maximale d'une page de résultats (100 lignes), imposée par l'endpoint records de l'API Explore v2.1.
- **_OFFSET_MAX** : offset maximum autorisé (10 000). Au-delà, la pagination s'arrête pour respecter la limite de l'API.

Méthodes :

- **__init__(self, timeout=30)** : constructeur de la classe. Il initialise _timeout (limite le temps d'attente de chaque requête HTTP) et _session (instance requests.Session partagée entre toutes les requêtes). Configure également un adaptateur HTTPAdapter avec retry automatique sur 3 tentatives, délai exponentiel (1s, 2s, 4s), déclenché sur erreurs réseau et codes HTTP 429/500/502/503/504.

Méthodes d'accès aux données — Effectifs :

- **get_effectifs(profession, departement_code, annee)** : reçoit une profession, un code département et une année. Interroge le dataset demographie-effectifs-et-les-densites avec un filtre ODSQL sur ces trois paramètres, tout âge et tout sexe. Retourne l'effectif et la densité pour l'année demandée.
- **get_evolution_effectifs(profession, departement_code, region_code=None)** : reçoit une profession et, selon le cas, un code département ou un code région. Interroge le même dataset sans filtre d'année pour récupérer toutes les années disponibles, triées chronologiquement. Retourne l'évolution des effectifs dans le temps.
- **get_effectifs_par_sexe(profession, departement_code, annee, region_code=None)** : reçoit une profession, un territoire (département ou région) et une année. Filtre les enregistrements sur tout âge et exclut la valeur agrégée tout sexe. Retourne la répartition des effectifs par sexe pour l'année demandée.
- **get_effectifs_par_age(profession, departement_code, annee, region_code=None)** : reçoit une profession, un territoire et une année. Filtre les enregistrements sur tout sexe et exclut la tranche Tout âge. Retourne la répartition des effectifs par tranche d'âge, triée par libellé de tranche.

Méthodes d'accès aux données — Exercice libéral :

- **get_exercices(profession, departement_code, annee, region_code=None)** : reçoit une profession, un territoire et une année. Interroge le dataset demographie-exercices-liberaux (qui utilise le champ code_departement et non departement). Retourne le type d'exercice libéral et l'effectif pour l'année demandée.
- **get_evolution_exercices(profession, departement_code, region_code=None)** : reçoit une profession et un territoire. Interroge le même dataset sans filtre d'année pour récupérer toutes les années disponibles, triées chronologiquement. Retourne l'évolution du type d'exercice libéral dans le temps.

Méthodes d'accès aux données — Secteurs conventionnels :

- **get_secteurs(profession, departement_code, annee, region_code=None, secteur_libelle=None)** : reçoit une profession, un territoire et une année. Accepte un filtre optionnel secteur_libelle (ex. Secteur 1, Secteur 2, Non conventionné). Interroge le dataset demographie-secteurs-conventionnels. Retourne le libellé du secteur et l'effectif.

Méthodes d'accès aux données — Pathologies :

- **get_pathology_labels(annee=None)** : reçoit une année optionnelle. Interroge le dataset effectifs avec un group_by sur patho_niv1, en filtrant sur tout âge, tout sexe, niveau prioritaire 1 et en excluant le code département 999. Retourne la liste triée des libellés de pathologies disponibles.
- **get_pathologies(annee=None, pathologie="all", region=None, departement=None, limit=5000)** : reçoit des filtres optionnels sur l'année, la pathologie, la région et le département. Interroge le dataset effectifs et agrège les résultats par clé (annee, region, departement, pathologie) en additionnant les nombres de patients et en calculant un taux de prévalence pondéré par la population. Retourne la liste triée par région, département et pathologie.

Méthodes d'accès aux données — Honoraires :

- **get_honoraires(profession, departement_code, annee)** : reçoit une profession, un code département et une année. Applique la correspondance de libellé via _CORRESPONDANCES_HONORAIRES si nécessaire. Interroge le dataset honoraires-detaillés avec un filtre ODSQL sur ces trois paramètres. Retourne les honoraires détaillés (type niveau 1, 2 et 3, montants) triés par ordre d'affichage.
- **get_evolution_honoraires(profession, departement_code)** : reçoit une profession et un code département. Filtre uniquement sur le type Ensemble des honoraires au niveau 1, sans filtre d'année. Retourne l'évolution du montant total des honoraires sur toutes les années disponibles, triée chronologiquement.
- **get_evolution_comparaison(profession, departement_1, departement_2, type_honoraires_niv1)** : reçoit une profession, deux codes département et un type d'honoraires de niveau 1. Interroge le dataset honoraires-detaillés pour les deux départements simultanément. Retourne les montants moyens par année et par département, triés par année et département, directement exploitables par Chart.js.
- **get_classement_departements(profession, annee)** : reçoit une profession et une année. Interroge le dataset honoraires-detaillés avec un group_by sur département et montant moyen (sans year() dans le select, ce qui évite le conflit API). Retourne la liste des départements triée par montant moyen décroissant.

Méthodes d'accès aux données — Médicaments :

- **get_medicaments(annee, poste_id=None, limit=50)** : méthode de diagnostic. Elle interroge le catalogue global de l'API pour lister les datasets disponibles contenant le mot prescriptions et affiche leurs identifiants dans le terminal. En attendant un dataset stable, elle retourne une liste vide pour éviter une erreur 503 et laisser le formulaire s'afficher correctement.

Méthode privée :

- **_requete(self, dataset, params)** : cette méthode reçoit le nom du dataset et un dictionnaire de paramètres de requête. Elle gère intégralement la pagination (pages de 100 enregistrements maximum, en incrémentant l'offset à chaque itération), le cache mémoire

TTL (vérifie la clé avant la requête, stocke le résultat en cas de succès) et le retry automatique configuré sur la session. La boucle s'arrête dès que l'une de ces trois conditions est remplie : le nombre total d'enregistrements demandé est atteint, l'API renvoie une page incomplète, ou l'offset dépasse `_OFFSET_MAX` (10 000). En cas d'erreur réseau ou HTTP, elle retourne None (et non une liste vide), ce qui permet aux contrôleurs de distinguer une panne API d'un résultat vide.

Utilitaires de cache :

- **cache_info()** : méthode statique qui retourne un dictionnaire contenant la taille actuelle du cache, la taille maximale et le TTL en secondes.
- **vider_cache()** : méthode statique qui vide entièrement le cache sous verrou (`_cache_lock`) et logue l'opération.

-grippe_api.py :

Ce fichier définit la classe GrippeAPI, qui permet d'accéder à l'api Odissé de santé publique France. Le dataset interrogé couvre l'activité grippale aux urgences et chez SOS Médecins par département et classe d'âge, sur environ 6 saisons (depuis 2020), Seule l'évolution dans le temps est exposée, ce qui permet de visualiser clairement les vagues hivernales.

Attribut de classe :

- **BASE_URL** : URL racine de l'API Odissé
- **DATASET** : identifiant du dataset grippe utilisé pour construire l'URL d'accès aux enregistrements.
- **_PAGE_MAX** : nombre maximum d'enregistrements récupérés par page
- **_OFFSET_MAX** : offset maximum autorisé (10 000), au-delà duquel la pagination s'arrête pour ne pas boucler indéfiniment.
- **CLASSES_AGE** : liste des tranches d'âge disponibles dans le dataset ("Tous âges", "00-04 ans", "05-14 ans", "15-64 ans", "65 ans ou plus").

Méthode de classe :

- **init(self, timeout=10)** : Constructeur de la classe. Il initialise deux attributs d'instance : `_timeout` utilisé pour limiter le temps d'attente de chaque requête http et `_session` une instance partagée entre toutes les requête de l'objet (évite de rouvrir une connexion TCP à chaque appel).
- **_requete(self, params)** : cette méthode reçoit un dictionnaire de paramètres de requête et gère intégralement la pagination. La méthode récupère les données en pages successives de 100 enregistrements maximum et découpe automatiquement en pages successives en incrémentant l'offset (le nombre d'enregistrement à sauter avant de commencer à envoyer des résultats) à chaque itération. La boucle s'arrête dès que l'une de ces trois conditions est remplie : le nombre total d'enregistrements demandé est atteint, l'API renvoie une page incomplète (signalant qu'il n'y a plus de données), ou l'offset dépasse `_OFFSET_MAX` (10 000). En cas d'erreur elle retourne les résultats déjà accumulés plutôt que de propager l'exception, ce qui permet de ne pas faire planter la page. Cette méthode retourne des enregistrement JSON.
- **get_evolution_departement(self, dep_code, classe_age)** : cette méthode reçoit le code du département et la classe d'âge sélectionnée par l'utilisateur et construit un filtre ODSQL (langage de filtrage propre aux APIs Opendatasoft) qui combine ces deux paramètres et demande 400 enregistrements maximum pour les trier par ordre chronologique. Elle retourne la liste des enregistrements, qui sera affichée sous forme de courbe multi-saisons dans la page espace.html.

-utilisateurs.py :

Ce fichier regroupe toute la logique liée aux mots de passe (hachage, vérification) et aux accès à la table utilisateur (recherche, création, authentification complète). Toutes ces fonctions sont appelées par le contrôleur auth.py.

- **Hacher_mot_de_passe(mot_de_passe)** : cette fonction reçoit un mot de passe en clair (chaîne de caractère) puis calcule son empreinte SHA-256. L'empreinte SHA-256 est sous forme de chaîne hexadécimale de 64 caractères.
- **Verifier_mot_de_passe(mot_de_passe, hache_stocke)** : cette fonction reçoit le mot de passe saisi en clair et le mot de passe haché récupéré dans la base pour un utilisateur. Elle recalcule le haché du mot de passe saisi, puis le compare à le haché stocké (SHA-256 étant déterministe,). Retourne un booléen : True si les deux hachés sont identiques, False sinon.
- **Chercher_par_identifiant(bdd,identifiant)** : cette fonction reçoit une session SQLAlchemy active et l'identifiant recherché. Elle interroge la table utilisateur avec un filtre sur identifiant et récupère le premier résultat correspondant. Puis elle retourne l'objet Utilisateur trouvé, ou None si aucun compte ne porte cet identifiant.
- **Creer_utilisateur(bdd,identifiant,mot_de_passe)** : cette fonction reçoit une session SQLAlchemy active avec l'identifiant choisi et le mot de passe en clair. Elle construit une nouvelle instance Utilisateur en hachant le mot de passe puis l'ajoute à la session (bdd.add) et la valide l'insertion (bdd.commit()). Elle retourne l'objet Utilisateur créé, avec son id généré automatiquement par MySQL.
- **Authentifier(bdd,identifiant,mot_de_passe)** : cette fonction reçoit une session SQLAlchemy active, l'identifiant et le mot de passe saisis au formulaire de connexion, puis enchaîne deux étapes : chercher le compte (avec chercher_par_identifiant()) puis s'il existe vérifier le mot de passe (avec verifier_mot_de_passe()). Elle retourne l'objet Utilisateur si les deux étapes réussissent sinon renvoie None.

controllers/

-accueil.py :

Ce fichier définit le Blueprint `bp_accueil`, qui regroupe les pages publiques de l'application. Il gère la page d'accueil avec son formulaire de sélection ainsi que les pages de présentation.

- **index() — route GET / et /accueil** : cette fonction de vue Flask affiche la page d'accueil. Elle ouvre une session SQLAlchemy, récupère la liste des régions (triée par libellé) et la liste des professions de santé (triée par libellé), puis retourne le template `accueil.html` avec ces deux listes transmises comme variables. La session est fermée dans un bloc `finally`.
- **acces_soins() — route GET /acces-soins et /acces-soins.html** : cette fonction de vue retourne simplement le template `acces-soins.html`. Elle ne reçoit et ne retourne aucun paramètre métier.
- **presentation() — route GET /presentation et /presentation.html** : cette fonction de vue retourne simplement le template `presentation.html`. Elle ne reçoit et ne retourne aucun paramètre métier.

-api.py :

Ce fichier définit le Blueprint `bp_api` (préfixe `/api`), qui expose des routes utilitaires en JSON pour alimenter l'interface front-end : listes de régions, de départements et statistiques de cache. Il instancie une unique instance de `AmeliAPI` réutilisée pour toutes les requêtes.

- **regions() — route GET /api/regions** : cette fonction de vue ouvre une session SQLAlchemy, récupère la liste des régions triées par code et retourne un tableau JSON contenant pour chaque région son id, son code et son libellé. La session est fermée dans un bloc `finally`.
- **departements(region_id) — route GET /api/departements/<int:region_id>** : cette fonction de vue reçoit un identifiant de région en paramètre d'URL. Elle ouvre une session SQLAlchemy, filtre les départements par `region_id`, les trie par code, et retourne un tableau JSON contenant pour chaque département son id (égal au code), son code et son libellé. La session est fermée dans un bloc `finally`.
- **cache_info() — route GET /api/cache/info** : cette fonction de vue retourne les statistiques actuelles du cache interne de `AmeliAPI` sous forme de JSON, via la méthode statique `AmeliAPI.cache_info()`.
- **cache_clear() — route POST /api/cache/clear** : cette fonction de vue vide entièrement le cache interne de `AmeliAPI` en appelant la méthode statique `AmeliAPI.vider_cache()`, puis retourne un JSON de confirmation avec le statut `ok`.

-auth.py :

Ce fichier définit le Blueprint `bp_auth`, qui regroupe toutes les routes liées à l'authentification et à l'espace personnel. Il gère trois fonctionnalités distinctes : la connexion, la création de compte et la page protégée. Toute la logique de sécurité est déléguée aux fonctions du fichier `services/utilisateurs.py`. Ce fichier intègre également le décorateur `login_required`, qui protège les routes réservées aux utilisateurs connectés.

- **login_required(vue)** : décorateur python (fonction qui modifie le comportement d'une vue). Il reçoit une fonction de vue Flask en paramètre et vérifie si la clé "utilisateur" est présente dans la session Flask, cette clé est ajoutée à la connexion et supprimée à la déconnexion. Si la clé est absente (utilisateur non connecté), il redirige immédiatement vers la route `/connexion` sans exécuter la vue.
- **connexion() — route GET/POST /connexion** : Cette fonction de vue Flask gère l'affichage et la soumission du formulaire de connexion. En GET, elle retourne simplement le template `connexion.html` sans erreur. En POST, elle récupère l'identifiant et le mot de

passer depuis request.form, ouvre une session SQLAlchemy et appelle authentifier(). Si l'authentification réussit, elle stocke l'identifiant de l'utilisateur dans la session Flask sous la clé "utilisateur" et redirige vers /espace. Si elle échoue elle affiche le template connexion.html avec un message d'erreur générique.

- **creer_compte() — route GET/POST /creer-compte** : cette fonction de vue Flask gère l'affichage et la soumission du formulaire de création de compte. En GET, elle retourne le template creer_compte.html sans erreur. En POST, elle récupère les trois champs du formulaire : identifiant, mot_de_passe et confirmation. Elle effectue deux validations avant toute interaction avec la base : vérification que l'identifiant et le mot de passe ne sont pas vides, et vérification que le mot de passe et sa confirmation sont identiques. Si une validation échoue, elle retourne le template avec un message d'erreur. Si les validations passent, elle ouvre une session SQLAlchemy et vérifie (avec chercher_par_identifiant()) qu'aucun compte n'existe déjà avec cet identifiant. Si un doublon est détecté, elle retourne le template avec une erreur. Sinon, elle appelle creer_utilisateur() et stocke l'identifiant dans la session Flask et redirige directement vers /espace : l'utilisateur est connecté immédiatement après la création de son compte sans avoir à se reconnecter.
- **deconnexion() — route GET /deconnexion** : cette fonction de vue supprime la clé "utilisateur" de la session Flask (avec flask_session.pop()) et redirige vers la page d'accueil.
- **espace() — route GET /espace** : cette fonction de vue flask est réservée aux utilisateurs connectés, protégée par le décorateur @login_required et reçoit deux paramètres optionnels via l'URL : departement_id et classe_age tous deux nécessaires pour filtrer les données grippales affichées dans le graphique en courbe.

-contact.py :

Ce module expose le Blueprint bp_contact avec deux routes gérant l'affichage et la soumission du formulaire de contact.

- **SUJETS** : dictionnaire Python mappant les clés HTML (bug, donnees, suggestion, autre) vers leurs libellés affichés à l'utilisateur dans la page de confirmation.
- **index() — route GET /contact** : affiche le formulaire de contact vierge en retournant le template contact.html. Ne reçoit et ne retourne aucun paramètre métier.
- **envoyer() — route POST /contact/envoyer** : reçoit les données du formulaire via request.form : nom, email, sujet et message (tous obligatoires, strip automatique). Vérifie que les quatre champs sont remplis. En cas de champ manquant, retourne contact.html avec une variable erreur. En cas de succès, logue le message en console (TODO : branchement Flask-Mail) et retourne le template contact_confirmation.html avec les informations saisies et le libellé du sujet récupéré depuis SUJETS.

-dashboard.py :

Ce fichier contient le Blueprint bp_dashboard avec trois routes principales. Il interroge l'API data.ameli.fr (prescriptions) et l'API ameli.fr (effectifs de professionnels de santé) via la classe AmeliAPI.

- **MOTS_MEDECIN** : liste de 19 mots-clés (médecin, généraliste, cardiologue...) utilisée pour détecter si un libellé de profession correspond à un médecin.
- **_est_medecin(libelle)** : reçoit un libellé de profession en chaîne de caractères. Teste si ce libellé contient au moins un mot-clé de MOTS_MEDECIN (insensible à la casse). Retourne un booléen.
- **_get_listes(session)** : reçoit une session SQLAlchemy active. Récupère en une seule fonction les trois listes de référence nécessaires aux formulaires de filtrage : régions, professions de santé et postes de prescription, tous triés. Retourne un tuple (regions, professions, postes).
- **_get_departements_par_region(session, region_id)** : reçoit une session SQLAlchemy active et un identifiant de région. Si region_id est absent, retourne une liste vide. Sinon,

interroge la table `Departement` filtrée par `region_id` et triée par `code`, puis retourne la liste des départements correspondants. Utilisée pour pré-remplir le select département du formulaire lorsqu'une région est déjà sélectionnée.

- **_filtrer_professions_prescription(professions)** : reçoit la liste complète des professions de santé récupérées en base. Filtre et retourne uniquement les professions dont le libellé ne figure pas dans `PROFESSIONS_PRESCRIPTION_EXCLUES`, c'est-à-dire les agrégats et catégories non pertinentes pour la visualisation des prescriptions.
- **medicaments() — route GET /medicaments** : affiche les données de prescription médicamenteuse pour une profession de santé donnée. Le département, le poste de prescription et l'année sont des filtres optionnels : si aucune année n'est renseignée, toutes les années disponibles sont retournées. Effectue deux requêtes vers l'API `data.ameli.fr` : l'une pour le tableau détaillé (filtrée selon les paramètres actifs), l'autre pour l'évolution historique toutes années confondues (filtrée uniquement par profession et, si présents, département et poste), utilisée pour le graphique. Retourne HTTP 503 si l'API ne répond pas. Paramètres query string : `profession_id` (obligatoire), `departement_id`, `poste_id`, `annee` (optionnels). Le paramètre `region_id` est lu mais n'est pas exploité dans le traitement des données de cette route.
- **pathologies() — route GET /pathologies** : page de visualisation des pathologies. Route simple retournant le template `pathologies.html`. Aucun paramètre accepté pour l'instant (données chargées côté client).
- **professionnels() — route GET /professionnels** : affiche les effectifs de professionnels de santé pour une profession, un territoire et une année donnés. Interroge quatre endpoints de l'API Ameli : effectifs bruts, évolution temporelle, répartition par sexe et répartition par tranche d'âge. Retourne HTTP 503 si l'API Ameli ne répond pas. Paramètres query string : `profession_id` et `annee` (obligatoires), `departement_id` ou `region_id` (au moins un requis).

-honoraire.py :

Ce fichier définit le Blueprint `bp_honoraire`, qui regroupe les routes liées à la visualisation des honoraires des professionnels de santé par département, à partir des données de l'API Ameli. Il expose une seule route `/honoraire` supportant deux visualisations distinctes : un tableau de classement des départements par montant moyen d'honoraires, et une courbe comparant l'évolution dans le temps de deux départements choisis par l'utilisateur. Une particularité de ce contrôleur est qu'il effectue lui-même la mise en forme des données brutes de l'API en une structure directement exploitable par `Chart.js` ce qui permet que le template `honoraire.html` n'a aucune transformation à effectuer.

Les constantes :

- **ANNEES_DISPONIBLES** : liste d'entiers allant de 2024 à 2015
- **TYPES_COMPARAIISON** : liste de quatre chaînes de caractères représentant les types d'honoraires proposés pour la visualisation en courbe : "Ensemble des honoraires", "Actes", "Dépassements" et "Rémunérations forfaitaires" (Correspond aux types disponibles dans le dataset Ameli).
- **VIZ_VALIDES** : ensemble Python contenant les deux valeurs de visualisation acceptées : "tableau" et "courbe". Utilisée pour valider le paramètre `viz_type` reçu depuis l'URL.
- **PROFESSIONS_EXCLUES_HONORAIRES** : ensemble Python contenant les libellés des professions à exclure du select, dont les données sont incomplètes ou non pertinentes dans le dataset honoraires (agrégats tels que "Ensemble des médecins", "Ensemble des auxiliaires médicaux", etc.).

Les fonctions :

- **_serie_comparaison(raw)** : cette fonction privée de mise en forme est appelée uniquement par la fonction `afficher()` lorsque la visualisation en courbe est demandée. Elle Reçoit la liste brute des enregistrements retournés par l'API construit et retourne un dictionnaire `{labels, series}` directement exploitable par `Chart.js`.
 - **labels** : liste des années disponibles dans les données, triées chronologiquement.

- **series** : une entrée par département, contenant son libellé et la liste de ses montants alignés sur labels
- **afficher() — route GET /honoraires et /honoraires.html** : fonction de vue principale et unique du Blueprint. Reçoit six paramètres optionnels via l'URL : `profession_id` , `departement_id` et `region_id` , `annee` , `viz_type` , `departement_id_2` et `region_id_2` , et `type_honoraires`. Elle ouvre une session SQLAlchemy pour récupérer les listes nécessaires aux formulaires. Elle n'appelle l'API Ameli que si les paramètres minimums sont remplis et uniquement pour la visualisation demandée (le tableau déclenche `api.get_classement_departements()` , la courbe déclenche `api.get_evolution_comparaison()` suivie de `_serie_comparaison()` pour préparer les données Chart.js). Retourne HTTP 400 si un `profession_id` ou `departement_id` fourni ne correspond à aucun enregistrement en base. Retourne le template `honoraires.html` avec l'ensemble des données.

-pathologies_api.py :

Ce fichier définit le Blueprint `bp_pathologies_api` (préfixe `/api`), qui expose les données de pathologies sous forme JSON. Il interroge le service AmeliAPI, puis enrichit les résultats avec les libellés des régions et des départements stockés en base de données. Il instancie une unique instance de AmeliAPI réutilisée pour toutes les requêtes.

- **_annotate_pathology_labels(rows)** : cette fonction privée reçoit la liste brute des enregistrements retournés par l'API. Elle ouvre une session SQLAlchemy, collecte les codes région et département présents dans les données, interroge les tables `Region` et `Departement` pour récupérer les libellés correspondants, puis les ajoute à chaque enregistrement sous les clés `region_libelle` et `departement_libelle`. La session est fermée dans un bloc `finally`. Retourne la liste enrichie.
- **pathologies() — route GET /api/pathologies** : cette fonction de vue reçoit quatre paramètres optionnels via l'URL : `year` (année filtrée), `pathologie` (libellé de pathologie, par défaut `all`), `region` (code région) et `departement` (code département), ainsi qu'un paramètre `distinct`. Si `distinct` est fourni, elle appelle `api.get_pathology_labels(annee=year)` et retourne directement la liste des libellés en JSON. Sinon, elle appelle `api.get_pathologies()` avec les filtres saisis, enrichit les résultats via `_annotate_pathology_labels()` et retourne le JSON final.

static/js/

-cascade.js :

Ce fichier gère la cascade région vers département sur la page d'accueil. Il écoute le changement du select region, vide la liste des départements quand la région change, appelle l'endpoint `/api/departements/<region_id>` via fetch et remplit le select departement avec les options reçues. En cas d'échec de la requête, il affiche un message d'erreur dans la liste. Il utilise la variable `BASE_URL` injectée par le serveur pour gérer le préfixe d'URL en déploiement sous-dossier.

-espace.js :

Ce fichier pilote le graphique de suivi de la grippe de la page Mon espace. Il lit le tableau d'enregistrements JSON injecté dans la page via un îlot script d'identifiant `data-evolution`, vérifie la présence du canvas d'identifiant `courbe-grippe` et de la librairie `Chart.js`, puis trace une courbe avec deux séries : le taux de passages aux urgences (`taux_passages_grippe_sau`) et le taux d'hospitalisations (`taux_hospit_grippe_sau`). L'axe X affiche un repère toutes les 13 semaines environ pour rester lisible sur environ 330 semaines. L'initialisation est lancée automatiquement au chargement de la page via `DOMContentLoaded` ou immédiatement si la page est déjà chargée.

-honoraires.js :

Ce fichier gère l'ensemble de la logique interactive de la page Honoraires. Il lit la configuration serveur (`baseUrl`, `vizType`) depuis un îlot JSON d'identifiant `page-config`. Il gère les onglets de visualisation (méthode `switchTab`), l'affichage ou le masquage des filtres de comparaison selon le type de vue sélectionné (méthode `majFiltresComparaison`), et le changement de visualisation (méthode `setViz`) qui soumet automatiquement le formulaire si profession et année sont déjà renseignées. Il branche la cascade région vers département via la méthode générique `brancherCascade`, utilisée à la fois pour le sélecteur principal et le sélecteur de comparaison secondaire. Il gère la pagination du tableau de classement (10 lignes par page, méthode `initClassement`) et dessine la courbe de comparaison des honoraires entre deux départements avec `Chart.js` (méthode `initCourbe`). L'initialisation complète est lancée via `DOMContentLoaded` ou immédiatement si la page est déjà chargée.

-pathologies.js :

Ce fichier pilote l'ensemble de la logique interactive de la page Pathologies. Il lit les URLs de l'API depuis l'objet `window.PATHOLOGIES_CONFIG` injecté par le serveur. Il gère trois modes de visualisation : tableau (`renderTable`), graphique en courbes par années (`renderChart`) et carte choroplèthe de France (`renderMap`). Il expose les fonctions `fetchPathologyLabels`, `fetchRegions`, `fetchDepartments` et `fetchPathologiesData` pour récupérer les données depuis les endpoints `/api/pathologies` et `/api/regions` et `/api/departements`. Il gère la cascade région vers département (`onRegionChange`), la pagination du tableau (15 lignes par page), l'affichage dynamique selon l'indicateur sélectionné (nombre de patients ou taux de prévalence) et le rendu de la carte via `Plotly` en mode région ou département. L'initialisation complète est lancée via `initializePathologies` au chargement de la page.

-prescription.js :

Ce fichier pilote la logique interactive de la page Prescriptions médicamenteuses. Il lit les données injectées par Flask depuis les variables globales `RESULTATS` et `EVOLUTION`. Il gère les onglets de la page (méthode `switchTab`), la bascule entre vue tableau et vue graphique (méthode `switchViz`), la cascade région vers département du formulaire (méthode `setupCascade`) et la pagination du tableau des prescriptions (15 lignes par page, méthodes `renderPage` et `initPrescriptionTable`). Il dessine le graphique d'évolution du montant total annuel des prescriptions

avec Chart.js (méthode initPrescriptionChart). L'initialisation complète est lancée via DOMContentLoaded.

-professionnels.js :

Ce fichier pilote la logique interactive de la page Professionnels de santé. Il lit les données injectées par Flask depuis les variables globales RESULTATS, EVOLUTION, PAR_SEXE et PAR_AGE. Il gère les onglets de la page (méthode switchTab), la bascule entre vue tableau et vue graphique (méthode switchViz), la cascade région vers département du formulaire (méthode setupCascade) et la pagination du tableau des effectifs (15 lignes par page, méthodes renderPage et initProfesTable). Il dessine trois graphiques avec Chart.js : un anneau de répartition par sexe (initSexeChart), un camembert de répartition par tranche d'âge (initAgeChart) et une courbe d'évolution de l'effectif et de la densité sur deux axes (initEvolutionChart). L'initialisation complète est lancée via DOMContentLoaded.

-table_export.js :

Ce fichier ajoute un bouton d'export CSV à chaque tableau présent dans la page. Il parcourt tous les éléments table du document, et pour chacun non encore traité (attribut data-csv-export-ready absent), crée un bouton Exporter en CSV inséré au-dessus du tableau. Au clic, il récupère toutes les lignes du tableau (via querySelectorAll tr et th/td), échappe les valeurs CSV (guillemets doublés, entourage entre guillemets), génère un fichier .csv avec un séparateur point-virgule et un BOM UTF-8, puis le propose au téléchargement. Le nom du fichier est construit à partir de l'attribut data-csv-filename du tableau et de la date du jour au format ISO.